

HEART RATE CALCULATOR

Complete VITyarthi Project Report

Student: Jyotishka Das

Course: Introduction to Problem Solving and Programming

University: VIT Bhopal

Date: 23/09/2026

Based on the student's original BPM calculator

1. Introduction

The Heart Rate Calculator is a simple Python command-line project. The original project calculates a person's heart rate in Beats Per Minute (BPM) using the number of heartbeats counted and the time taken to count them. The original project uses the formula $\text{BPM} = (\text{number of beats} / \text{time in seconds}) \times 60$. The extended version keeps this formula and adds modular design, validation, classification, storage and testing.

2. Problem Statement

A user who counts their heartbeats for a measured number of seconds needs to convert that count into Beats Per Minute. Doing the calculation manually is simple but repetitive. The proposed Python program accepts the values, applies the BPM formula and presents the result clearly.

3. Objectives

- Calculate BPM from heartbeat count and measurement time.
- Implement the mathematical formula using Python.
- Create a command-line application that can be executed directly.
- Validate user input.
- Provide a simple interpretation of the calculated BPM range.
- Store calculation history.
- Demonstrate modular Python programming and testing.

4. Scope

The project is an educational command-line calculator. It accepts manually counted heartbeats and time in seconds. It calculates BPM, classifies the result into simple ranges and saves the calculation. It does not measure heart rate using a sensor and does not diagnose medical conditions.

5. Target Users

- Students learning Python and problem solving.
- Beginners learning functions, conditions, loops and file handling.
- Users who need a simple BPM calculation from a manual count.

6. Technologies Used

Programming language: Python 3.x

Interface: Command Line / Terminal

Storage: JSON

Version control: Git / GitHub

Testing: pytest-compatible test files

External libraries: None required for the main program.

7. Formula Used

$\text{BPM} = (\text{Number of Beats} / \text{Time in Seconds}) \times 60$

Example:

$\text{BPM} = (60 / 60) \times 60$

$\text{BPM} = 60 \text{ BPM}$

8. Functional Requirements

- FR1 - The system shall accept the user's name.
- FR2 - The system shall accept the number of heartbeats counted.
- FR3 - The system shall accept measurement time in seconds.
- FR4 - The system shall validate that beats are not negative and time is greater than zero.
- FR5 - The system shall calculate BPM using the original project formula.

FR6 - The system shall classify BPM as below 60, 60-100, or above 100 BPM.
FR7 - The system shall display the result and calculation.
FR8 - The system shall save the calculation to JSON storage.
FR9 - The system shall provide automated tests for calculation and classification.

9. Non-Functional Requirements

Usability – prompts and output should be simple for beginners.

Performance – the calculation should complete immediately for normal input.

Reliability – invalid values should not cause the program to crash.

Maintainability – different responsibilities should be placed in separate modules.

Portability – the main program should run with Python 3.x without external packages.

Resource Efficiency – the application should use minimal memory and a small JSON file.

Error Handling – invalid integer input and invalid ranges should be handled.

10. System Architecture

The application follows a simple modular architecture. `heart_rate_calculator.py` acts as the main controller. `validator.py` handles input validation, `calculator.py` handles the BPM formula, `classifier.py` handles the simple range classification, `display.py` presents the result, `history.py` handles JSON storage, and `config.py` stores constants.

See `docs/system_architecture.md` for the architecture diagram.

11. Use Case Diagram

The main actor is the User. The user enters a name, heartbeat count and time. The system validates the values, calculates BPM, classifies the result, displays the report and saves the calculation.

See `docs/use_case.md` for the use-case diagram.

12. Workflow Diagram

The workflow is: Start → Enter Name → Validate Name → Enter Beats → Enter Time → Validate Values → Calculate BPM → Classify BPM → Display Result → Save Result → End.

See `docs/workflow.md` for the workflow diagram.

13. Sequence Diagram

The user provides input to the main controller. The controller asks the validator to check input, sends valid values to the calculation module, obtains the BPM, sends it for classification, displays the result and saves it using the history module.

See `docs/sequence.md` for the sequence diagram.

14. Class / Component Diagram

The project is function-based rather than class-based. Its components are:

- main controller - `heart_rate_calculator.py`
- calculation - `calculator.py`
- validation - `validator.py`
- classification - `classifier.py`
- presentation - `display.py`
- storage - `history.py`
- configuration - `config.py`

See `docs/component.md` for the component diagram.

15. Database / Storage Design

The project does not use a relational database. It uses data/history.json. Each stored record contains name, beats, time_seconds, heart_rate_bpm and category. Since there are no relational tables, an ER diagram is not applicable; a JSON storage design is provided instead.

See docs/storage_design.md.

16. Design Decisions and Rationale

The original formula and command-line approach were retained so the extended project remains consistent with the student's existing work. The code was divided into small files to meet the modularity requirement without making the logic difficult to understand. JSON was selected because it is simple and requires no database setup. The original BPM range messages were retained as the classification basis.

17. Implementation Details

The program starts from heart_rate_calculator.py. The validator obtains valid user input. calculator.py performs $(\text{beats} / \text{time}) \times 60$. classifier.py assigns one of three simple categories. display.py prints the result and the formula. history.py saves the result in JSON. Automated tests verify calculations and classification.

18. Sample Output

Example based on the original project:

enter your name: xyz

enter the number of heartbeats counted: 60

enter the time in seconds: 60

heart rate: 60.0 BPM

Formula:

```
heart rate = (60 / 60) × 60
heart rate = 60.0 BPM
```

19. Testing Approach

The test suite checks several meaningful cases:

- 60 beats in 60 seconds gives 60 BPM.
- 30 beats in 30 seconds gives 60 BPM.
- 75 beats in 60 seconds gives 75 BPM.
- 50 beats in 45 seconds gives approximately 66.67 BPM.
- 59 BPM is classified below 60 BPM.
- 60 and 100 BPM are classified in the 60-100 BPM range.
- 101 BPM is classified above 100 BPM.

Run tests with pytest if it is installed.

20. Challenges Faced

- Keeping the original program simple while adding project-level requirements.
- Separating one script into meaningful modules.
- Handling invalid input without stopping unexpectedly.
- Saving results without introducing a database.
- Creating documentation and design diagrams.

21. Advantages

- Simple and easy to use.
- Runs directly from the command line.
- Does not require external libraries for the main program.
- Provides a quick BPM calculation.
- Useful for learning basic Python programming.
- Now includes validation, storage, modular design and testing.

22. Limitations

- The program depends on the accuracy of user input.
- It does not measure heart rate using a sensor.
- It is not intended for medical diagnosis.
- The range classification is a simple educational feature and does not account for individual medical circumstances.

23. Future Enhancements

- Add a graphical user interface.
- Connect the program to a heart-rate sensor or wearable device.
- Add charts and visualizations.
- Add CSV export.
- Add more detailed history and date/time records.
- Add a sensor-data simulation mode.

24. Learnings and Key Takeaways

The project demonstrates user input, variables, arithmetic operations, conditions, loops, exception handling, functions, modules, JSON file handling, testing, documentation and GitHub organization. It also demonstrates how a simple Python program can be expanded into a more complete software project without changing its core formula.

25. Conclusion

The Heart Rate Calculator successfully calculates heart rate in Beats Per Minute using the original BPM formula. The extended project keeps the student's original command-line idea while adding the missing software-project elements required by VITyarthi: functional modules, non-functional requirements, modular code, input validation, storage, testing, diagrams, GitHub documentation and a detailed report.

26. References

- Python 3 documentation: <https://docs.python.org/3/>
- VITyarthi - Build Your Own Project instructions supplied for this assignment.
- Student's original Heart Rate Calculator code, project executability document and project report.

27. Safety / Educational Note

This is an educational programming project. It calculates BPM from manually entered heartbeat count and time. It is not a medical device and should not be used to make medical decisions.

Appendix A – Original Project Alignment

The original project asks for the user's name, heartbeat count and time in seconds, then calculates BPM using the formula $(\text{beats/time}) \times 60$ and displays the result. The complete version keeps these elements and adds the required project structure around them.

Appendix B – File List

File	Purpose
heart_rate_calculator.py	Main program
calculator.py	BPM calculation
validator.py	Input validation
classifier.py	BPM classification
display.py	Output/report display
history.py	JSON storage
config.py	Constants
tests/test_calculator.py	Calculation tests
tests/test_validator.py	Classification tests
README.md	GitHub documentation
statement.md	Project statement